

Test API Execution Observations

Paylocity Benefits Dashboard

Notable findings that are not formal bugs — for developer and product owner awareness

May 21, 2026 · Diego Lizarraga

Observations

001 Username always assigned from authenticated user credentials, not request body

Every employee created through the API has username and partitionKey set to the authenticated user's username (TestUser970), regardless of what was submitted in the request body.

This appears to be an intentional architecture decision since employees are saved or partitioned by authenticated user. However it contradicts the swagger schema which defines username as a required client-supplied field.

```
required": [
  "firstName",
  "lastName",
  "username"
],
```

Automation implication

The created_employee fixture in pytest will always see TestUser970 as the username. Tests asserting username should assert the authenticated user's username, not the submitted value.

002 DELETE returns 200 with empty response body which is acceptable behavior

Successful DELETE operations return 200 OK with an empty response body. This is acceptable REST behavior. Some APIs return 204 No Content which is arguably more semantically correct, but 200 with empty body is not a bug, although it is preferable.

Automation implication

Delete assertions should check status=200 only. Don't assert on response body content for delete operations.

003 sortKey field in response is identical to id field

Every employee response includes both id and sortKey fields, and both contain the same UUID value. This suggests sortKey is a DynamoDB implementation detail that was accidentally exposed in the public API response.

```
{ "id": "5ebaaaae-053b-4fcd-8d69-49ea5ccb211d",
  "sortKey": "5ebaaaae-053b-4fcd-8d69-49ea5ccb211d", ... }
```

Recommendation

sortKey and partitionKey are internal NoSQL database implementation details and should not be exposed in the public API response. They reveal the underlying data store architecture (DynamoDB).

004 ReadOnly fields gross/net/benefitsCost/partitionKey correctly ignored in POST and PUT

TC-API-POST-025 and TC-API-PUT-009 confirmed the following fields are correctly ignored when submitted by the client:

- gross — server always computes from salary
- benefitsCost — server always computes from business rules
- net — server always computes from gross and benefitsCost
- partitionKey — always set to authenticated username

Salary is NOT ignored — see BUG-API-003. This is the critical exception.

This is a passing result for gross, net, benefitsCost, and partitionKey. The API correctly protects these computed values from client manipulation.

005 405 pattern across multiple endpoints for invalid input

The following scenarios all return 405 Method Not Allowed with an empty body instead of appropriate error codes:

- POST with string in dependants field — should be 400
- POST with float in dependants field — should be 400
- PUT without ID in body — should be 400
- PUT with non-existent ID — should be 404
- DELETE non-existent ID — should be 404
- DELETE malformed UUID — should be 400 or 404
- GET malformed UUID — should be 400 or 404

This pattern suggests the API routing layer isn't handling edge cases correctly, it falls through to a 405 when it can't match a route due to unexpected input.

Likely a framework-level routing issue rather than individual validation bugs.

I recommend validating with product owner to confirm intended behavior.

006 salary field behavior requires product owner clarification

The salary field exists in the API schema and is client-settable (confirmed by BUG-API-003). The business rules document states all employees earn \$2,000 per paycheck, which corresponds to \$52,000 annually.

Open questions for product owner

- Is salary intended to be fixed at \$52,000 for all employees?
- If fixed, salary should be marked readonly in the swagger schema.
- If variable, the business rules document needs updating.

The current state — salary is settable but undocumented as such — is the worst of both worlds.

007 SQL injection strings accepted as literal strings → positive security finding

API POST requests with SQL injection strings in firstName and lastName fields:

- ' OR '1'='1 — accepted as literal string, no 500, no data leakage
- '; DROP TABLE employees;-- — accepted as literal string, table intact after submission
- admin'-- in username — overwritten with TestUser970 per BUG-API-002

The API correctly handles injection attempts as literal strings..